Sistemas Operativos
Tarea 1
Planificación de Tareas

ÍNDICE

EL SIMULADOR 1
Batch Processing 3
Time Sharing 4
PREGUNTAS 5

El objetivo de la tarea es el uso de un simulador de ejecución de tareas en Batch Processing y en tiempo compartido.

Cuidado

La tarea es un trabajo individual. La tarea se entrega usando el sistema *docencia.informaticae.org* con una fecha y hora límite que deben respetar. Si por una razón X no pueden entregar la tarea a tiempo, deben avisar por mail. Si al momento de entregar la tarea el servidor docencia no funciona (y solamente en este caso), pueden entregar la tarea por mail.

EL SIMULADOR

El archivo **tarea1-tar.gz** contiene un simulador de planificación de tareas capaz de simular una ejecución en modo Batch Processing o Time Sharing. Deben recuperar el archivo tar y extraerlo en una carpeta (el archivo tar contiene una sub-carpeta **tarea1-so**):

```
> cd micarpeta
> tar xfz tarea1-so.tar.gz
```

Deben compilar la tarea en la sub-carpeta **Debug**:

```
micarpeta> cd Debug
micarpeta/Debug> make clean
micarpeta/Debug> make all
```

Es imperativo usar el commando **make clean** para evitar problemas entre diferentes arquitecturas de procesadores. El simulador se llama **tarea1-so** y toma 2 ó 3 argumentos en función del tipo de simulación:

- Modo **Batch Processing:** El primer argumento es 1 y el segundo argumento es el archivo de definición de tareas:

```
> tarea1-so 1 archivo.txt
Ejemplo
> tarea1-so 1 t1.txt
```

- En modo **Time Sharing:** El primer argumento es "2", el segundo corresponde al valor del quantum de tiempo en ms, y el tercero es el archivo de definición tareas.

```
> tarea1-so 2 quantum archivo.txt
Ejemplo
> tarea1-so 2 10 t1.txt
```

Nota
Los ejemplos están en la misma carpeta **Debug**

El loop principal del simulador corresponde a la función *RunCPU()*. Cada iteración de esta función corresponde a una unidad de tiempo. Un quantum de $10ms$, corresponde entonces a 10 iteraciones de este loop. El loop se termina cuando todas las tareas han terminado.

Un archivo de definición de tareas tiene el siguiente formato (se debe respetar el formato exacto):

```
CANTIDAD_DE_TAREAS
LLEGADA  DURACION CANTIDAD_E/S  <TIEMPO_E/S:DURACION>...
```

Ejemplo:

```
>cat t1.txt
6
0 200 3 10:30 45:20 110:25
0 50 1 5:10
50 100 2 30:5 40:5
100 15 0
20 40 1 20:5
300 20 0
```

Hay 6 tareas en total. Los tiempos de ejecución no incluyen los tiempos de E/S (no es el tiempo de respuesta). Los tiempos de llegada son tiempos absolutos. Los tiempos de E/S son tiempos relativos al momento de recuperar el procesador después de una E/S.

- La tarea T_0 llega al instante $t = 0$ y dura $200ms$ (sin incluir sus E/S), hace 3 entradas/salidas:
  - La primera al instante $t = 10ms$ que dura $30ms$.
  - La segunda $45ms$ después de recuperar el procesador cuando se termina la E/S anterior, y con una duración de $20ms$.

  - La última $110ms$ después de recuperar el procesador cuando se termina la E/S anterior y con una duración de $25ms$.

- La tarea T_1 llega al instante $t = 0$, dura $50ms$ y tiene una sola E/S que dura $10ms$ después de $5ms$ de ejecución.
- La tarea T_2 llega a $t = 50ms$, dura $100ms$ y hace dos E/S de $5ms$
  - La primera después de $30ms$ de ejecución.
  - La segunda $40ms$ después de recuperar el procesador después que se termine la primera E/S.
- La tarea T_3 llega a $t = 100ms$ y dura $15ms$. No hace ninguna E/S.
- La tarea T_4 llega a $t = 20ms$ y hace una sola E/S de $5ms$, después de $20ms$ de ejecución.
- La tarea T_5 llega a $t = 300ms$, dura $20ms$ y no hace ninguna E/S.

El archivo **refman.pdf** (carpeta latex) corresponde a una documentación mínima del código.

BATCH PROCESSING

El simulador entrega el siguiente resultado para una ejecución en modo Batch:

```
./tarea1-so 1 t2.txt
Tasks simulator v1.1: BATCH PROCESSING

Task 0: start at: 0, duration: 200
IOs at: 10:30  45:20  110:25  35:END
Task 1: start at: 0, duration: 50
IOs at: 5:10  45:END
Task 2: start at: 50, duration: 100
IOs at: 30:5  40:5  30:END
Task 3: start at: 100, duration: 15
IOs at: 15:END
Task 4: start at: 20, duration: 40
IOs at: 20:5  20:END
Task 5: start at: 300, duration: 20
IOs at: 20:END
0000000000#XXXXXXXXXXXXXXXXXXXXXXXXXXXXXX000000000000000000000000000
00000000000000000#XXXXXXXXXXXXXXXXXXX000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000
000000000000000#XXXXXXXXXXXXXXXXXXXXXXXXX000000000000000000000000000
0000000000*/11111#XXXXXXXXXX1111111111111111111111111111111111111111
111111*/22222222222222222222222222222#XXXXX22222222222222222222222
2222222222222222#XXXXX22222222222222222222222222*/333333333333
333*/44444444444444444444#XXXXX44444444444444444444*/55555555555555
555555*/

Tasks: 6, Total execution time: 525
Total CPU idle time: 100 (19.05%)
```

```
Task 0, CPU time: 200, Elapsed: 275
Task 1, CPU time: 50, Elapsed: 335
Task 2, CPU time: 100, Elapsed: 395
Task 3, CPU time: 15, Elapsed: 360
Task 4, CPU time: 40, Elapsed: 485
Task 5, CPU time: 20, Elapsed: 225
```

La primera parte corresponde a las tareas que se van a ejecutar. La ejecución de las tareas corresponde a:

- El número de la tarea que se ejecuta si tiene el procesador.
- Un X si no hay ninguna tarea lista para ejecutarse.
- Un # cuando la tarea hace una entrada-salida.
- Un / cuando hay conmutación de tarea.
- Un $*$ cuando la tarea se termina.

Al final el simulador entrega estadísticas de ejecución:

- La cantidad de tareas ejecutadas.
- El tiempo total de ejecución.
- El tiempo ocioso (idle) del procesador.

Y para cada tarea:

- El tiempo de uso del procesador (que debe corresponder a la duración de la tarea en el archivo de definición de tareas).
- El tiempo de respuesta de la tareas (Elapsed).

TIME SHARING

En modo Time Sharing, el simulador entrega la siguiente información:

```
./tarea1-so 2 10 t1.txt
Tasks simulator v1.1: TIME SHARING

Task 0: start at: 0, duration: 200
IOs at: 10:30  45:20  110:25  35:END
Task 1: start at: 0, duration: 50
IOs at: 5:10  45:END
Task 2: start at: 50, duration: 100
IOs at: 30:5  40:5  30:END
Task 3: start at: 100, duration: 15
IOs at: 15:END
Task 4: start at: 20, duration: 40
IOs at: 20:5  20:END
Task 5: start at: 300, duration: 20
IOs at: 20:END
0000000000#/11111#/XXXXX4444444444/1111111111/4444444444#/0000000000/1111111111/2222222222/4444444444/0000000000/1111111111/2222222222/3333333333/4444444444*/0000000000/1111111111/2222222222#/33333*/0000000000/11111*/2222222222/00000#/2222222222/2222222222/0000000000/2222222222#/0000000000/2222222222/0000000000/2222222222/0000000000/2222222222*/5555555555/0000000000/5555555555*/0000000000/0000
```

```
000000/0000000000/0000000000/0000000000/0000000000#/XXXXXXXXXXXXXX
XXXXXXXXXX0000000000/0000000000/0000000000/00000*/

Tasks: 6, Total execution time: 455
Total CPU idle time: 30 (6.59%)
Task 0, CPU time: 200, Elapsed: 455
Task 1, CPU time: 50, Elapsed: 190
Task 2, CPU time: 100, Elapsed: 255
Task 3, CPU time: 15, Elapsed: 75
Task 4, CPU time: 40, Elapsed: 120
Task 5, CPU time: 20, Elapsed: 35
```

La primera parte corresponde a las tareas que se van a ejecutar. La ejecución de las tareas corresponde a:

- El número de la tarea que se ejecuta si tiene el procesador.
- Un X si no hay ninguna tarea lista para ejecutarse.
- Un / cuando se termina el quantum de tiempo.
- Un # cuando ocurre una entrada/salida.
- Un $*$ cuando la tarea se termina.

Al final el simulador entrega las estadísticas de ejecución.

PREGUNTAS

Ejecute al archivo t2.txt con el simulador en modo Batch:

```
> tarea1-so 1 t2.txt
```

Pregunta 2.1
¿Explicar por qué la ejecución del mismo ejemplo en modo Time Sharing afecta bastante el tiempo de respuesta de la tarea T_1?

Pregunta 2.2
Proponer un ejemplo de archivo de definición de tareas, con 3 tareas en total, donde la ejecución en modo Batch y Time Sharing produce el mismo tiempo de respuesta para todas las tareas. ¿Explicar por qué?
Deben incluir este archivo en el TAR de respuesta.

Pregunta 2.3
Proponer un ejemplo de archivo de definición de tareas, con 4 tareas, donde todas la tareas se benefician de la ejecución en tiempo compartido, excepto la primera tarea que llega al tiempo $t = 0$.
***Deben incluir este archivo en el tar de respuesta**.*

Considera el siguiente código de la función *UpdateTasks()*:

```
void UpdateTasks(unsigned time, unsigned n)
{
unsigned i;

for(i=0; i<n; i++)
{
        switch( tasks[i].state)
        {
        case ready:
                tasks[i].rtime++;
                break;
        case blocked:
                tasks[i].rtime++;
                tasks[i].run[tasks[i].index].io--;
                if (tasks[i].run[tasks[i].index].io == 0)
                {
                tasks[i].index++;
                tasks[i].state = ready;
                }
                break;
        case notready:
        case running:
        default: break;
        }
}
}
```

Pregunta 2.4
Explicar el rol de la función UpdateTasks(). Se pide comentar la función completa.

El siguiente código es parte de la función de ejecución de las tareas en modo Time Sharing:

```
rdy = CheckTaskReady(tasks, time, n);
if (rdy > 0)
{
        // Run the next task
        current_task = TimeShareResched(tasks, current_task, n);
        used_quantum = 0;
        index = tasks[current_task].index;
        // Run task for at most one quantum
        ...
```

Pregunta 2.5
¿Qué mecanismo necesario para el Time Sharing implementa la función TimeShareResched()?

La función *TimeShareResched()* tiene el siguiente código:

```
/**
* \brief Chose next task to execute (shechdule) in time sharing. \n
* Uses a simple Round Robin policy without preemption, and without priorities.
* The function is called at end of quantum or at IO
* \param[in] tasks The tasks list
* \param[in] current_task The current task in execution
* \param[in] n The number of tasks
* \return The next task to run
* \return -1 If there is no more tasks
* \return -2 If there are tasks in state notready or blocked and none is available to run
*/
int TimeShareResched(task_t tasks,int current_task, unsigned n)
{
        unsigned count=0;
        unsigned wait=0;
        int index;

        if (current_task == -2) index = -1;
        else index = current_task;

        while( count<n )
        {
                index = (index+1)%n;
                count++;
                if (tasks[index].state == ready)
                {
                        tasks[index].state = running;
                        return(index);
                }
                if ((tasks[index].state == blocked) || (tasks[index].state == notready)) wait++;
        }
        if (wait != 0) return(-2);
        else return(-1);
}
```

Pregunta 2.6
¿Qué algoritmo implementa esta función? Justificar.

Pregunta 2.7
¿Qué funciones del código habría que modificar para implementar un Time Sharing con gestión de prioridades (tipo Multi Levels Queues)?
Se pide explicar de forma general las modificaciones, sin escribir código.